

Deep Learning

4. Loss functions, scaling

Romain Tavenard

Optimization in general

- **Goal:** Tune model parameters so as to minimize error
- Measuring error
 - Through a cost function / loss function
 - Typical example: Mean Squared Error in regression settings

Loss functions

- To train a model, we need to **measure how wrong it is**:
that's the loss function $\mathcal{L}$
- Standard losses
 - **Mean Squared Error (MSE)** for regression
(penalizes large errors)

$$\mathcal{L}(x_i, y_i; \theta) = (m(x_i; \theta) - y_i)^2$$

- **Cross-entropy** for classification
(penalizes confident wrong predictions)

$$\mathcal{L}(x_i, y_i; \theta) = -\log P_{\theta}(y = y_i \mid x_i)$$

- Technical requirement for gradient descent:
 $\mathcal{L}$ must be differentiable

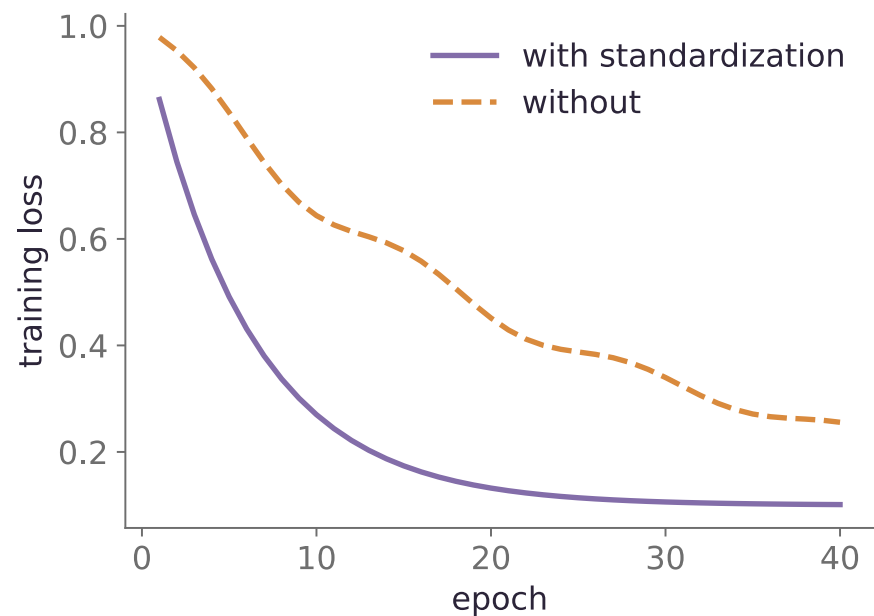
Data preprocessing

- In practice, **scaling input features** is critical for stable training

- ▶ Recommendation: center-reduce each feature

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}$$

- ▶ or use domain-specific scaling
(e.g. images in $[0, 1]$)



Weight initialization

- **Intuition:** keep signal “loudness” (variance) **constant** across layers
 - too large $\rightarrow$ activations explode
 - too small $\rightarrow$ fade to 0 (vanishing gradients, *cf* next course)
- **Where numbers come from:** neuron sums n_{in} inputs $\rightarrow$ variance $\times n_{\text{in}}$ $\rightarrow$ to keep it stable:

$$\text{Var}(w) = \frac{1}{n_{\text{in}}}$$

Weight initialization

Two classic schemes = refinements of $\text{Var}(w) = 1/n_{\text{in}}$:

- **Xavier / Glorot** (tanh, sigmoid):

$$w \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$$

- ▶ stable forward (n_{in}) **and** backward (n_{out}) $\rightarrow$ harmonic-mean
- ▶ assumes φ linear near 0
- **He** (ReLU):

$$w \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right)$$

- ▶ ReLU kills **half** the signal $\rightarrow$ **double** the variance (factor 2)

Batch Normalization

- **Init fixes scale only at $t = 0$** $\rightarrow$ drifts as weights update
- **Batch Normalization:** Same goal as init, **enforced during training**: re-standardize layer inputs **every** mini-batch

$$\hat{z}^{(l)} = \frac{z^{(l)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}$$

- then learnable scale γ + shift β

Conclusion

- Loss functions
 - Mean Squared Error for regression
 - Logistic loss for classification
- Keep activations well-scaled
 - input standardization
 - init → scale at start
 - BatchNorm → scale during training